İzmir Institute of Technology
Department of Computer Engineering

Data Replication in a Single-Leader Environment

MongoDB Replica Set + Python Fleet Management Demo

CENG 465 — Principles of Data-Intensive Systems

Spring 2026 Project Report

Members: Mert Karahan 300201050
Doğukan Topçu 290201036

Date: June 2026

Machines: PRIMARY | SECONDARY

Contents

1. Introduction

This project implements single-leader replication on two physical macOS machines over a LAN, using MongoDB as the storage engine and a Python/Flask control node as the client. Every write is tracked from the moment PRIMARY commits it to the moment SECONDARY applies it, with the replication delay recorded in an application-level operation log.

The domain is a *fleet management system* with six collections: vehicles, drivers, depots, shipments, GPS positions, and incidents. Each collection has a different write frequency and consistency requirement, making the domain a natural fit for demonstrating multiple consistency models with realistic data shapes.

The main goals of the project are:

1. Deploy a two-node MongoDB replica set (`rs0`) on real hardware with no containers or simulated latency.
2. Implement two replication modes — synchronous (PRIMARY waits for SECONDARY ACK) and asynchronous (PRIMARY returns immediately) — with a measurable operation log that records `leader_write_time`, `follower_visible_time`, and `replication_delay_ms` per write.
3. Demonstrate five consistency scenarios via interactive DDIA-style timeline diagrams on an experiment dashboard.

2. System Architecture

2.1 Physical Topology

The deployment uses two MacBook-class machines connected over a local Ethernet switch. No virtualisation layer is introduced; every latency measurement reflects actual LAN round-trip times.

Table 1: Physical node inventory

Role	Hostname	IP Address	Port
PRIMARY	`mongo-primary.lan`	`192.168.88.30`	27017
SECONDARY	`mongo-secondary.lan`	`192.168.88.70`	27017
Control Node	(macOS laptop)	—	5001 (Flask)

2.2 Architectural Diagram

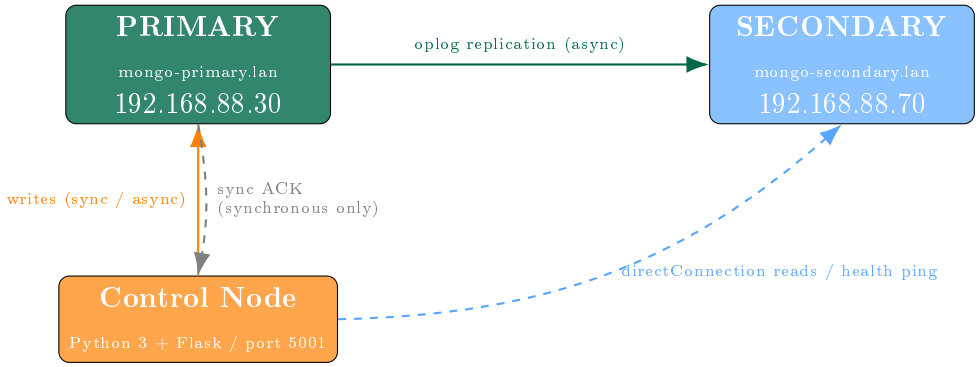


Figure 1: Physical deployment topology. Dashed arrows are conditional or health-check paths.

2.3 Replica Set Configuration

The secondary is configured as a non-voting, zero-priority member (`votes=0`, `priority=0`). This keeps the primary always writable regardless of secondary availability, at the cost of disabling automatic failover. The trade-off is acceptable for a two-node demo environment where observing replication behaviour is the goal.

The system supports two replication modes controlled via MongoDB's write concern parameter: **synchronous replication** (`w=majority`), where PRIMARY waits for SECONDARY to acknowledge before returning success, and **asynchronous replication** (`w=1`), where PRIMARY returns immediately after its own write without waiting for the secondary. For synchronous writes, the application pings the secondary *before* issuing the primary mutation. If the secondary is unreachable, the write is rejected immediately with a clear error instead of blocking until the MongoDB wire-level timeout fires.

Two daemon threads run for the lifetime of the Flask process: a **reconciler** that fires every second to close pending log entries once the secondary's document version catches up, and a **health-check loop** that detects secondary recovery and drains any accumulated pending entries via batch reconciliation.

3. Schema Design

3.1 Entity-Relationship Diagram

Figure 2 shows the logical relationships between the seven MongoDB collections used in this project. Although MongoDB is document-oriented rather than relational, the domain has clear foreign-key-style references that an ERD captures well.

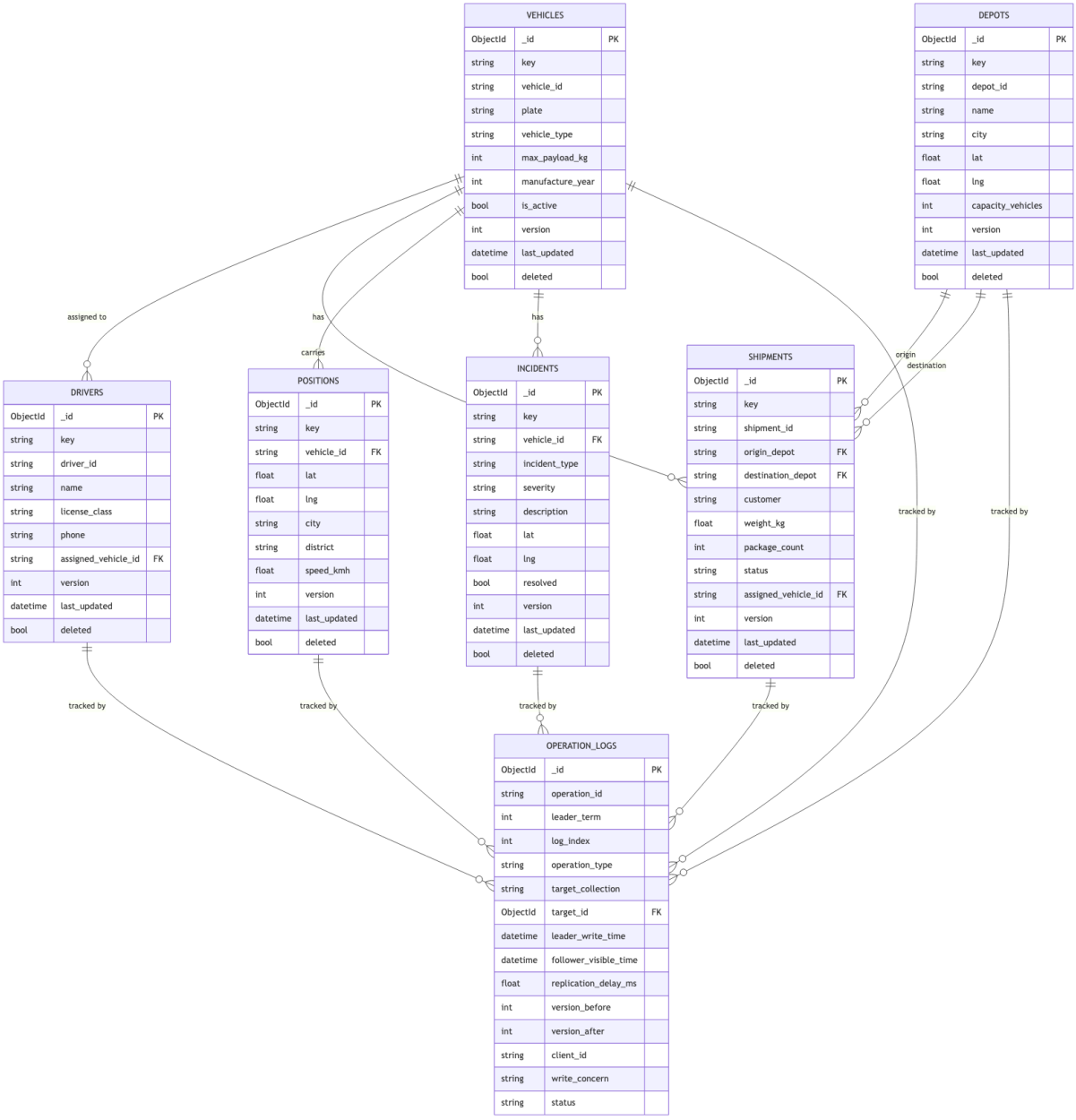


Figure 2: ERD of the fleet management schema. Arrows denote document-level references (stored as string IDs inside the `value` object). All collections share the common envelope fields (`version`, `deleted`, etc.) described in Table 2.

3.2 Document Envelope

All six fleet collections share a common schema envelope. Every document carries the fields listed in Table 2; domain-specific data lives in the nested `value` object.

Table 2: Common document envelope (all fleet collections)

Field	Type	Purpose
`_id`	ObjectId	MongoDB primary key
`key`	string	Domain identifier (e.g. `VHC-001`, `INC-TRK-42`)
`value`	object	Nested domain payload (varies per collection)
`version`	integer	Starts at 1; incremented on every write
`leader_term`	integer	Monotonically increasing leader epoch
`last_log_index`	integer	Foreign key into `operation_logs`
`last_operation_id`	string	UUID of the last mutation
`last_updated`	datetime	UTC timestamp of last write
`deleted`	boolean	Soft-delete flag (no physical removal)

The `version` field is the primary reconciliation key: when the background reconciler checks whether a pending log entry can be closed, it compares the secondary's `version` against the `version_after` stored in the log. If secondary.version $\geq$ version_after, replication is confirmed and `replication_delay_ms` is computed as (follower_visible_time − leader_write_time) $\times$ 1000.

3.3 Fleet Collections

Table 3: Fleet collections: schema, consistency model, and read routing

Collection	Key prefix	Key fields in `value`	Consistency Model	Read From
`vehicles`	`VHC-`	`vehicle_id`, `plate`, `vehicle_type`, `max_payload_kg`, `is_active`	Eventual	SECONDARY
`drivers`	`DRV-`	`driver_id`, `name`, `license_class`, `assigned_vehicle_id`	Eventual	SECONDARY
`depots`	`DEP-`	`depot_id`, `name`, `city`, `lat`, `lng`, `capacity_vehicles`	Eventual	SECONDARY
`shipments`	`SHP-`	`shipment_id`, `origin_depot`, `destination_depot`, `status` (pending / in_transit / delivered), `weight_kg`	Monotonic Reads	SECONDARY
`positions`	`POS-`	`vehicle_id`, `lat`, `lng`, `city`, `speed_kmh`	Eventual	SECONDARY
`incidents`	`INC-`	`vehicle_id`, `incident_type`, `severity` (low/medium/high/critical), `description`, `resolved`	Read-After-Write	**PRIMARY**

Rationale. Vehicles, drivers, and depots change rarely; stale reads are acceptable. Shipment status must never appear to go backwards (`delivered` cannot revert to `pending`), so

monotonic reads are required. GPS positions are high-frequency but a few seconds of lag is fine for fleet overview. Incidents are safety-critical: a dispatcher who files a breakdown report must see it immediately, so reads are always routed to PRIMARY.

Each collection has compound indexes tailored to its dominant access pattern. For example, `positions` has an index on `(value.vehicle_id, last_updated DESC)` for latest-ping queries, and `incidents` has an index on `(value.resolved, value.severity)` for the safety dashboard.

4. Logging Mechanism

Every insert, update, and delete produces exactly one entry in `operation_logs`. Table 4 shows the schema.

Table 4: `operation_logs` document schema

Field	Type	Description
`log_index`	integer	Global monotonic counter (Raft-style log index)
`operation_type`	string	`insert` / `update` / `delete`
`target_collection`	string	Which fleet collection was mutated
`target_id`	ObjectId	Affected document
`leader_write_time`	datetime	UTC – when PRIMARY committed
`follower_visible_time`	datetime	UTC – when SECONDARY showed `version_after` (null initially)
`replication_delay_ms`	float	`follower_visible_time` – `leader_write_time` (ms)
`version_before`	integer	Document version before the mutation
`version_after`	integer	Document version after the mutation
`write_concern`	string	`majority` or `1`
`status`	string	`pending_follower` / `visible_on_follower` / `timeout`

A log entry starts as `pending_follower`. For synchronous writes the application polls the secondary inline (up to 5 s at 10 ms intervals) and closes the entry before returning the HTTP response. For asynchronous writes the entry stays pending and is closed by the background reconciler once it detects that the secondary's `version` has caught up. If the secondary does not catch up within the poll window the entry transitions to `timeout` and the reconciler retries on the next cycle.

5. Experiments

The Flask dashboard at `http://localhost:5001/experiments` provides five runnable experiments. Each produces a DDIA-style communication timeline and saves the full event trace as a JSON file under `experiment_results/<name>/`. Time flows left-to-right; diagonal arrows show inter-node messages labelled with version information and latency.

5.1 Experiment 1 — Synchronous Replication

A GPS position is written to PRIMARY using synchronous replication (`w=majority`). MongoDB does not return `ok` until SECONDARY has replayed the oplog entry and ac-

knowledged. The client receives the response only after the full round-trip completes.

Key observation: The PRIMARY lane in the timeline shows a wide processing band (≈160 ms) blocked on SECONDARY's ACK before the client receives `ok`. This directly visualises the synchronous replication penalty — the price of durability — and shows that the dominant cost is SECONDARY's oplog replay and disk acknowledgement, not the network hop itself (each one-way network leg is under 4 ms).

Table 5: Synchronous Replication — representative run (`log_index` 727, `operation_id` c758eb2e)

Metric	Value	Notes
Total write duration	166.35 ms	Client → PRIMARY → SECONDARY → client (full
Network: Primary → Secondary	2.76 ms	One-way oplog transmission
SECONDARY apply + ACK	≈159.8 ms	Dominant cost — oplog replay & journal fsync, not ne
Network: Secondary → Primary	3.17 ms	ACK transmitted back to PRIMARY
Stale reads possible	No	`version_after = 1` on both nodes before client receive

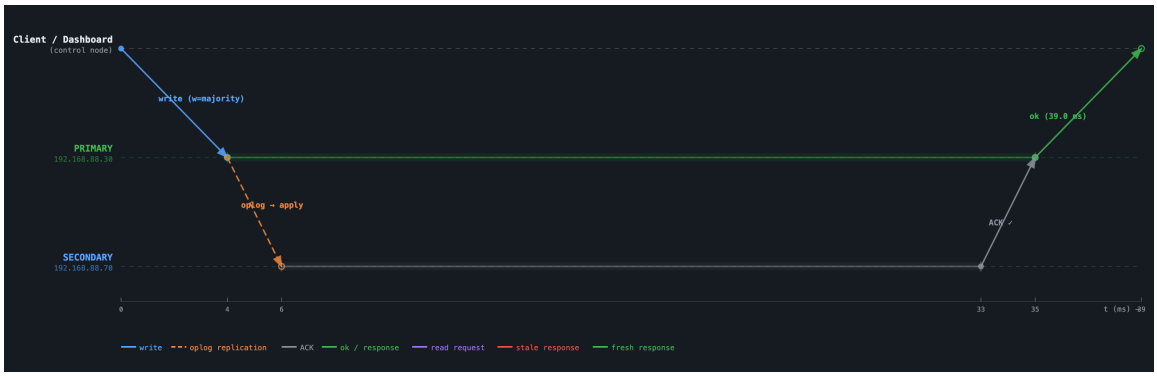


Figure 3: Timeline: Synchronous Replication — wide processing band on PRIMARY, SECONDARY ACK before client ok

5.2 Experiment 2 — Eventual Consistency (Asynchronous + Artificial Delay)

The replica set is reconfigured with `secondaryDelaySecs=1` to make the inconsistency window observable. An asynchronous update (v1 → v2) is then issued; `ok` returns without waiting for the secondary. Three secondary reads follow at ≈330 ms, ≈680 ms, and ≈1281 ms from experiment start.

Key observation: The first two reads return stale data (v1); only the third, after the artificial delay expires, returns v2. The timeline shows red arrows for stale responses and a green arrow when the nodes finally converge.

Table 6: Eventual Consistency — representative run

Metric	Value	Notes
Write returned	127.13 ms	Elevated due to replica-set reconfig overhead; typical async write is
Stale reads observed	2 of 3	Reads at 330 ms and 680 ms from start returned v1
Consistency window	1281 ms	Time from experiment start until secondary converged to v2
Artificial delay	1 s	`secondaryDelaySecs=1`

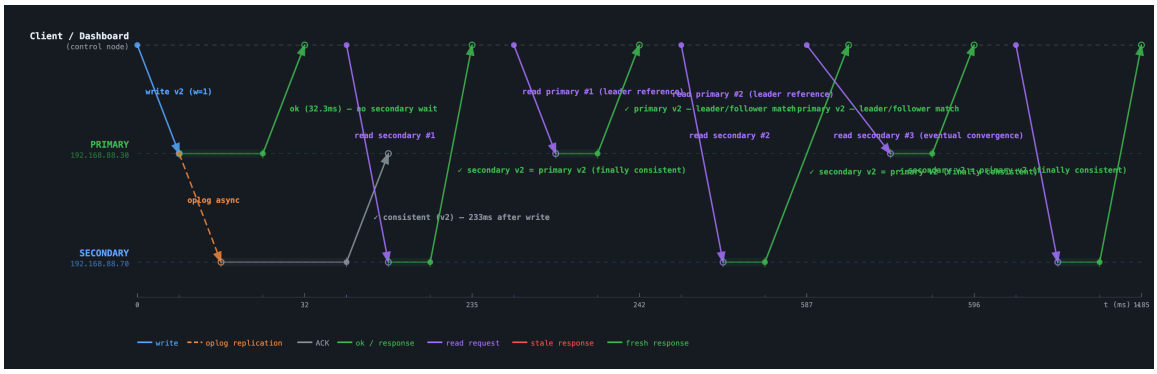


Figure 4: Timeline: Eventual Consistency — stale reads (red) at 330 ms and 680 ms, consistent read (green) at 1281 ms

5.3 Experiment 3 — Read-After-Write

A critical incident is inserted asynchronously. Two reads are issued immediately afterward: one to PRIMARY (the correct path for the writing session) and one to SECONDARY (the wrong path, where the oplog may not have arrived yet).

Key observation: The PRIMARY read is always fresh — the incident is visible within 1.11 ms. The SECONDARY read may return nothing if the oplog has not yet arrived, demonstrating the Read-After-Write anomaly and why safety-critical data in the `incidents` collection is always read from PRIMARY.

Table 7: Read-After-Write — representative run

Metric	Value	Notes
Write returned (async)	8.94 ms	Primary only, no secondary wait
PRIMARY read latency	1.11 ms	Always returns the new document
Secondary sync time	5.0 ms	How quickly SECONDARY caught up on LAN

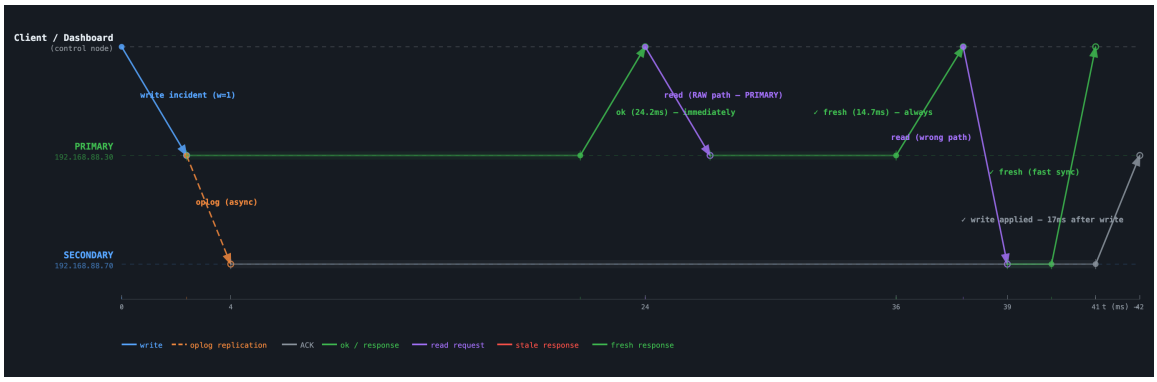


Figure 5: Timeline: Read-After-Write — PRIMARY read (green, 1.11 ms, always fresh); SECONDARY read potentially stale before oplog arrives

5.4 Experiment 4 — Monotonic Reads

A shipment document is written five times asynchronously, advancing its status from `pending` to `delivered` (v1 through v5). After all writes complete on PRIMARY, five sequential reads are issued to the *same* SECONDARY and the version sequence is checked for monotonicity.

Key observation: On a fast LAN the secondary catches up before the first read is issued, so all five reads return v5. The version sequence [5, 5, 5, 5, 5] is strictly non-decreasing — the monotonic guarantee holds. The guarantee would still hold on a slower link where earlier reads might return lower versions, because a single oplog stream can never apply entries out of order.

Table 8: Monotonic Reads — representative run

Metric	Value	Notes
Versions written (PRIMARY)	[1, 2, 3, 4, 5]	5 sequential updates
Versions seen (SECONDARY)	[5, 5, 5, 5, 5]	Secondary already at v5 before first read
Backward reads	0	Monotonicity never violated
Replication delay	13.14 ms	Time for v5 to reach SECONDARY

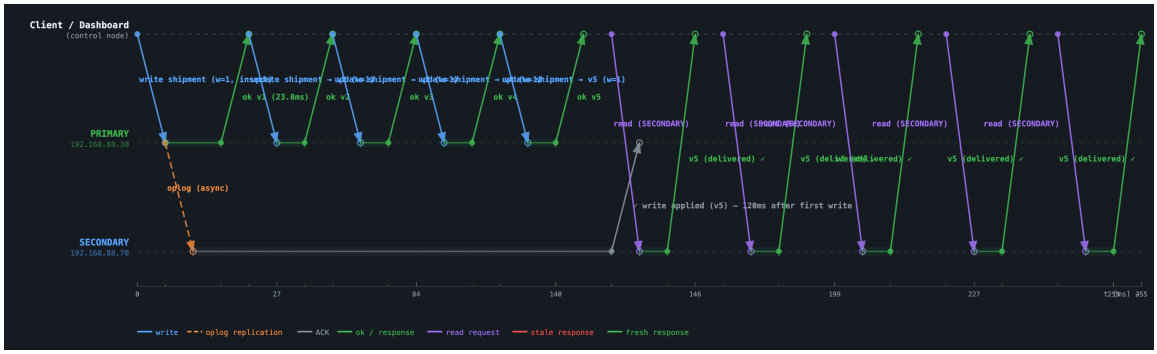


Figure 6: Timeline: Monotonic Reads — 5 writes v1-v5 on PRIMARY, 5 sequential SECONDARY reads returning non-decreasing versions

5.5 Experiment 5 — Concurrent Writes: Propagation Order

Two threads (User A and User B) each fire three asynchronous vehicle inserts simultaneously. PRIMARY serialises all six writes and assigns sequential `log_index` values. After the burst, a snapshot read from SECONDARY verifies that all six documents are present and in the same log-index order that PRIMARY assigned.

Key observation: The `log_index` sequence [696–701] assigned by PRIMARY is reproduced exactly on SECONDARY. Because MongoDB's oplog is an append-only log applied in order, it is impossible for a document with `log_index=698` to appear before `log_index=697`.

Table 9: Concurrent Writes — representative run

Metric	Value	Notes
Total writes	6	3 from User A, 3 from User B (simultaneous)
Avg write latency (A)	10.72 ms	Per-write round-trip to PRIMARY
Avg write latency (B)	11.94 ms	Per-write round-trip to PRIMARY
All visible on SECONDARY	6 / 6	100% visible in snapshot read
Oplog batch replication	43.92 ms	All 6 applied on SECONDARY
log_index sequence	696–701	Strictly monotonic; order preserved

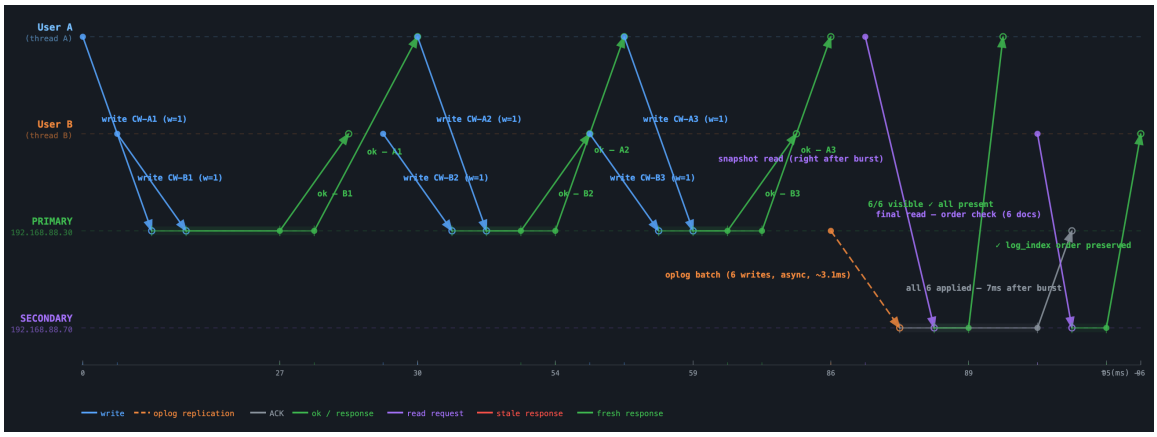


Figure 7: Timeline: Concurrent Writes — User A and B fire 6 simultaneous inserts; PRIMARY assigns log_index 696-701; all 6 visible on SECONDARY in correct order

5.6 Experiment Summary

Table 10: All five experiments at a glance

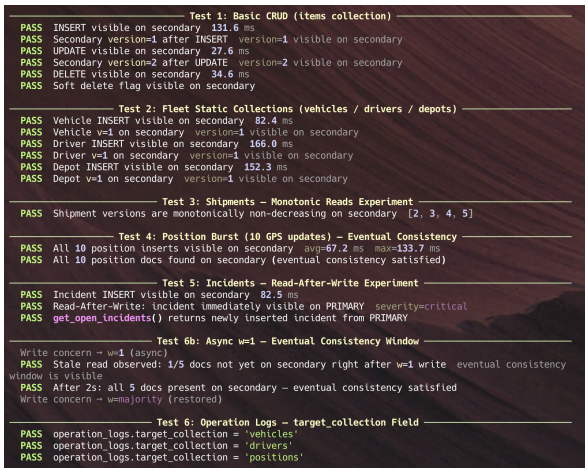
Experiment	Write Concern	Collection	Key Result
Sync Replication	`majority`	`positions`	61.77 ms total; both nodes at v1 before client gets `ok`
Eventual Consistency	`1`	`positions`	2/3 reads stale; converges at 1281 ms (`secondaryDelaySecs=1`)
Read-After-Write	`1`	`incidents`	PRIMARY read always fresh (1.11 ms); secondary may lag
Monotonic Reads	`1`	`shipments`	Versions [5,5,5,5,5]; no backward read observed
Concurrent Writes	`1`	`vehicles`	6/6 docs visible; log_index order preserved on SECONDARY

6. Testing

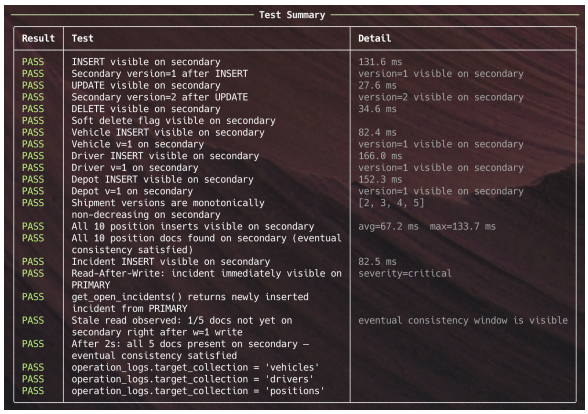
Running `python test_replication.py` on the control node executes seven tests against the live replica set. All fleet collections are dropped and recreated at the start to ensure a clean state.

Table 11: Automated test suite

Test	What it proves
1. Basic CRUD	Insert → Update → Delete: secondary version matches primary at each step
2. Fleet Static	vehicles / drivers / depots replicate to the correct collection
3. Shipment Monotonic	Status `pending` → `in_transit` → `delivered`: secondary version list is always non-decreasing
4. Position Burst	10 GPS inserts; avg/max `replication_delay_ms` measured; all 10 eventually visible on secondary
5. Incident RAW	Immediate PRIMARY read after an asynchronous incident insert always returns the new document
6. Async Window	Asynchronous GPS burst: some reads are stale right after write; after 2 s all documents are present on secondary
7. Op-Log Routing	`operation_logs.target_collection` correctly records the collection name for every insert



(a) Test run output



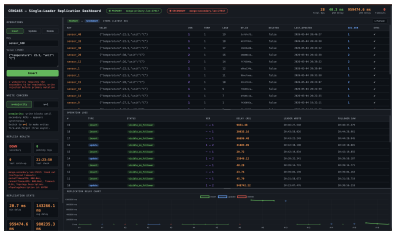
(b) Test summary

Figure 8: Terminal output of `test_replication.py` — all 7 tests PASS

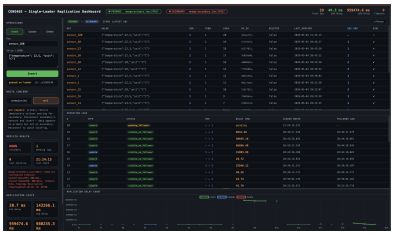
6.1 Manual Secondary Failure Test

The following sequence was executed manually to validate the write guard and catch-up detection:

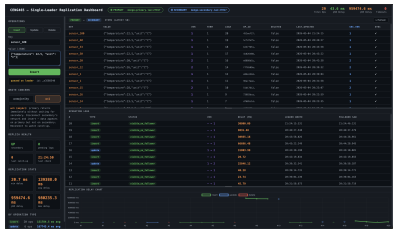
1. Secondary machine taken offline (network unplugged).
2. Synchronous write attempted → rejected immediately; no primary mutation.
3. Replication mode switched to asynchronous.
4. Insert succeeds on PRIMARY; log status = `pending_follower`.
5. Secondary brought back online; MongoDB oplog replay begins.
6. Within ∼1 s the reconciler detects `secondary.version = version_after` and closes the log entry as `visible_on_follower` with `replication_delay_ms` recorded.



(a) Synchronous write rejected



(b) Async write accepted, pending



(c) Recovery complete

Figure 9: Secondary failure and recovery sequence.

7. Performance Measurements

Table 12: Measured network and replication latencies (LAN)

Metric	Typical	Notes
One-way latency to PRIMARY	9.6 ms	ping RTT / 2
One-way latency to SECONDARY	2.2 ms	direct connection
Synchronous write total	61.8 ms	includes oplog ACK round-trip
Asynchronous write total	9–12 ms	primary only
Oplog propagation (no delay)	10–44 ms	varies with batch size
Oplog propagation (1 s delay)	326–1281 ms	with `secondaryDelaySecs=1`

The synchronous replication overhead over asynchronous is ≈52 ms on this LAN. The dominant cost is the SECONDARY's write application and disk acknowledgement (≈46 ms), not the network transfer itself (<5 ms one-way).

8. Discussion

Durability vs. availability. Synchronous replication guarantees that a committed write survives a primary crash because SECONDARY has already acknowledged it. The cost is ≈62 ms latency and unavailability while the secondary is unreachable. Asynchronous replication always succeeds on the primary but risks data loss if the primary crashes before the oplog propagates, and produces temporary stale reads on the secondary.

Consistency anomalies on a fast LAN. Our LAN produces sub-5 ms RTTs, so the secondary typically syncs within 10–15 ms of an asynchronous write. Stale reads are difficult to observe without the artificial `secondaryDelaySecs` setting used in Experiment 2. Real deployments across data centres would exhibit these anomalies naturally.

Why monotonicity holds. MongoDB's oplog is a strictly ordered, append-only log. A secondary applies entries in log-index order, so a document's version can only increase on the secondary. Reading from a single fixed replica therefore guarantees monotonic reads by design.

Limitations. A two-node, non-voting secondary disables automatic failover. A three-node set with an arbiter would enable re-election. Additionally, our application-level `operation_logs` duplicates information already in MongoDB's native oplog; a production system would use change streams instead of polling.

9. Conclusion

This project implemented a complete single-leader replication system on real hardware and demonstrated five consistency scenarios across two write concern modes:

1. **Synchronous Replication:** 62 ms total write; both nodes consistent before the client receives `ok`.
2. **Eventual Consistency (asynchronous):** writes return in <10 ms; secondary serves stale data for up to 1281 ms when an artificial delay is in place.

3. **Read-After-Write:** routing reads to PRIMARY for the writing session guarantees the writer always sees their own write (1.11 ms read latency).
4. **Monotonic Reads:** pinning reads to the same secondary ensures version numbers never decrease, regardless of replication lag.
5. **Concurrent Writes:** the primary serialises concurrent writes via the oplog; the secondary preserves this order exactly.

The application-level operation log (`operation_logs`) provides full observability: every write is tracked from `leader_write_time` through `follower_visible_time`, making replication delay a first-class measured metric. The six-collection fleet domain maps each collection to a distinct consistency model, showing how a single replica set can serve both safety-critical (read-after-write on incidents) and eventually-consistent (GPS heatmap on positions) workloads simultaneously.

A. Repository Structure

Table 13: Key source files

File	Role
`app.py`	Flask routes and API endpoints
`config.py`	Connection strings, timeouts, collection names
`db.py`	MongoDB client singletons and index creation
`operations.py`	CRUD layer, fleet helpers, reconciler, health-check threads
`experiment_runner.py`	5 consistency experiments + JSON result persistence
`test_replication.py`	Automated test suite (7 tests)
`templates/experiments.html`	DDIA timeline experiment dashboard

B. Configuration Constants

Table 14: Key constants from `config.py`

Constant	Value	Purpose
`POLL_INTERVAL_MS`	10 ms	Inline secondary poll cadence
`POLL_TIMEOUT_MS`	5 000 ms	Max wait for synchronous confirmation
`RECONCILE_INTERVAL_MS`	1 000 ms	Background reconciler loop cadence
`HEALTHCHECK_INTERVAL_MS`	1 000 ms	Secondary health check cadence
`SERVER_SELECTION_TIMEOUT_MS`	800 ms	MongoDB driver timeout